

# CSCI 232

## 232 Algorithms & Data Structures

LECTURE: 4

J. L. Pach

# OUTLINE:

- GitHub basics
- Fork
- Core Workflow

# GITHUB

# WHAT IS GITHUB?

- A **cloud-based platform** for hosting Git repositories.
- Provides **collaboration tools**: pull requests, issues, code review.
- Integrates with **Continuous Integration (CI) / Continuous Delivery (CD)**, project management, and documentation.
- Owned by **Microsoft** since 2018.

# GIT VS GITHUB

- **Git** = version control system (local).
- **GitHub** = remote hosting + collaboration features.
- GitHub uses Git under the hood.

# GITHUB AUTHENTICATION METHODS

- **HTTPS**:
  - Login with **username + password** (deprecated for push).
  - Now requires **Personal Access Token (PAT)** instead of password.
- **SSH**:
  - Secure authentication using **SSH keys**.
  - Recommended for frequent use.
- **GitHub CLI** (gh)

# AUTHENTICATION — HTTPS ONLY

- In this course we use **HTTPS only**
- Every `push` requires:
  - **username** — your GitHub login
  - **password** — a **Personal Access Token (PAT)**, not your account password
- No SSH setup required on lab machines.

# SSH — IT EXISTS, BUT NOT NOW

- GitHub also supports **SSH key** authentication
- More secure and convenient for frequent use
- For this course — **we are not setting up SSH**, HTTPS is sufficient

# HOW TO CREATE A NEW REPOSITORY ON GITHUB 1/2

1. **Log in** to your GitHub account.
2. Click the **"+" icon** in the top-right corner.
3. Select **"New repository"**.
4. Enter a **Repository name** (e.g., my-project).
5. (Optional) Add a **Description**.
6. Choose **Public** or **Private** visibility.

# HOW TO CREATE A NEW REPOSITORY ON GITHUB 2/2

7. (Optional) Initialize with:
   - **README.md**
   - **.gitignore**
   - **License**
8. Click **Create repository**.
9. Copy the **SSH or HTTPS URL** to clone locally.

# FROM MASTER TO MAIN – WHY THE CHANGE? 1/2

**Background:**

- Historically, Git's default branch was named **master**.
- In recent years, the term was considered **politically and socially insensitive** in some contexts.

**GitHub's Response:**

- GitHub changed the **default branch name** to **main** for new repositories.
- This is part of an **inclusive language initiative** across the tech industry.

# FROM MASTER TO MAIN – WHY THE CHANGE? 2/2

**What Does This Mean for You?**

- When creating new repositories on GitHub, the default branch will be **main**.
- If your local repo still uses **master**, you can rename it:

```
git branch -m master main
```

# HTTPS VS SSH — DO NOT MIX UP THE LINKS!

Every repository has **two different URLs**:

`HTTPS: https://github.com/user/repo.git`

`SSH: git@github.com:user/repo.git`

**If you configured HTTPS — use the HTTPS link.**

**If you configured SSH — use the SSH link.**

> Mixing these links = authentication error.
>
> Most common mistake at the start — remember it now.

# GIT

## clone

# GIT CLONE — WHAT DOES IT ACTUALLY DO?

```
git clone https://github.com/YOUR-LOGIN/repo.git
```

- Downloads the entire repository from the remote server **to your disk**
- Creates a folder named after the repository
- Automatically sets up `origin` — remembers where you cloned from
- After clone you have the **full commit history** locally

# THE CORE WORKFLOW

```
git clone https://github.com/YOUR-LOGIN/repo.git # 1. Clone your fork (once only)

cd repo                                          # 2. Enter the folder

        # 3. Modify the code in your editor

git status                                  # 4. Check what changed

git add .                                   # 5. Add files to the staging area

git commit -m "implement bubble sort"                  # 6. Commit with a description

git push                                         # 7. Send to GitHub
```

# STAGING AREA — WHY THIS EXTRA STEP?

```
[files on disk] → git add → [staging area] → git commit → [history]
```

- `git add` — tell Git: **include these files in the next commit**
- `git commit` — **save a snapshot** of the staging area into history
- Lets you modify many files but commit **selectively**

```
git add sort.c      # single file
git add .           # all modified files
git status          # always check before committing
```

# FORK

# WHAT IS A FORK?

- **Fork** = your own copy of someone else's repository on `GitHub`
- Changes in your fork **do not affect** the original repository
- In this course: each assignment comes as a repository link
- You **fork** → **clone** → **work locally** → **push to your fork**



# HOW TO FORK A REPOSITORY

1. Go to the assignment repository page
2. Click the **Fork** button (top-right corner)
3. Select your account as the destination
4. GitHub creates a copy under your account
5. Copy the **HTTPS link** to your fork

> You clone **your fork**, not the instructor's repository.

# THE FULL PICTURE




- `clone` — once, at the beginning
- `add → commit → push` — after every meaningful change

# COMMON MISTAKES

| Mistake | Cause |
| --- | --- |
| `Permission denied` on `push` | Used SSH link with HTTPS configuration |
| `remote: Invalid username or password` | PAT expired or typed account password |
| `nothing to commit` | Forgot `git add` before `git commit` |
| `Push works but CI sees no changes` | Cloned instructor's repo instead of your fork |

# SUMMARY

1. **Fork** the assignment repository on GitHub
2. **Clone** your fork locally
3. Work locally: **modify** → **add** → **commit**
4. **Push** to your fork
5. CI checks your code automatically

> One cycle: `git add .` → `git commit -m "..."` → `git push`
> Master this — you will do it every single class.

# THANK

**You!**